esprit Se former autrement HONORIS UNITED UNIVERSITIES	Année Universitaire : 2025-2026 **Applications Web Distribuées** **UP_WEB**

Application multi-conteneurs

Mise en situation

- Dans le cas d'une application qui répond à une montée en charge et dans le cas de plusieurs instances de chaque Microservice, il devient nécessaire d'automatiser les tâches de paramétrage, de builds des images et de run des différentes instances, de définir les dépendances entre les microservices…

- Docker-Compose est un outil permettant de définir et d'exécuter des applications Docker multi-conteneurs. Avec Docker-Compose, on utilise un fichier YAML pour configurer les servicesde notre application. Ensuite, avec une seule commande, nous créons et démarrons tous les services à partir de cette configuration.

- Docker Compose permet en effet d'orchestrer les conteneurs, et ainsi de simplifier les déploiements sur de multiples environnements. Docker Compose est un outil écrit en Python qui permet de décrire, dans un fichier YAML, plusieurs conteneurs comme un ensemble de services.

Objectifs

- Créer un fichier **docker-compose.yml** et écrire le script docker-compose.

- Lancer l'application multi-conteneurs **localement**

- Pousser l'application multi-conteneurs vers **le Hub** puis la lancer sur **le cloud** en utilisant l'environnement cloud temporaire **KillerCoda.**

Etape 1: Ecrire le script de configuration

Dans un fichier nommé **docker-compose.yml**, ajouter le script d'automatisation. Ce script permet de faire le build, le run … et d'autres configurations relatives aux conteneurs Docker. Ci-après un exemple de script qui fait le build des 2 microservices déjà développés dans l'atelier précédent (candidate-service et job-service) ainsi que le discovery-service, l'Api gateway et le serveur de configuration. Il fait aussi le run d'une instance de chacun des service

```
version: '3.9'

services:

  config-server:
    build: ./ConfigServer5SE4
    image: inesmjd/configserver:1.0
    container_name: configServer5se4
    ports:
      - "8888:8888"
    depends_on:
      - eureka-server
    environment:
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka

  job:
    build: ./Job
    image: inesmjd/job:1.0
    container_name: job5se4
    ports:
      - "8081:8081"
    depends_on:
      - mysqldb
      - eureka-server
      - config-server
    environment:
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
      -
SPRING_DATASOURCE_URL=jdbc:mysql://mysqldb:3306/jobTest?createDatabaseIfNotExist=true&useSSL=
false&max_allowed_packet=15728640
      - SPRING_DATASOURCE_USERNAME=root
      - SPRING_DATASOURCE_PASSWORD=root

  candidat:
    build: ./Candidat5SAE2
    image: inesmjd/candidat:1.0
    container_name: candidat5se4
    ports:
```

```
      - "8082:8080"
    depends_on:
      - eureka-server
      - config-server
    environment:
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888

  eureka-server:
    build: ./EurekServer5SE4
    image: inesmjd/eureka:1.0
    container_name: eureka5se4
    ports:
      - "8761:8761"
    environment:
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka

  API-gateway:
    build: ./ApiGateway5SE4
    image: inesmjd/apigateway:1.0
    container_name: gateway5se4
    ports:
      - "9999:9999"
    depends_on:
      - eureka-server
      - config-server
      - candidat
      - job
    environment:
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
      -
SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI=http://localhost:8099/realms/Job_Board_R
ealm_ApprocheA
      -
SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_JWK_SET_URI=http://keycloak:8099/realms/Job_Board_R
ealm_ApprocheA/protocol/openid-connect/certs


  mysqldb:
    image: mysql:5.6
    container_name: db
    ports:
      - "3306:3306"
    restart: unless-stopped
    environment:
      - MYSQL_ROOT_PASSWORD=root
      - MYSQL_DATABASE=jobTest
```

```yaml
keycloak:
  image: quay.io/keycloak/keycloak:26.3.1
  container_name: keycloak
  restart: always
  command: start-dev --http-port=8099
  environment:
    - KC_BOOTSTRAP_ADMIN_USERNAME=admin
    - KC_BOOTSTRAP_ADMIN_PASSWORD=admin
    - KC_HOSTNAME_STRICT=false  #désactive la vérification stricte du hostname et autorise
                    #les requêtes provenant d'autres hôtes (comme localhost depuis Postman)

  ports:
    - "8099:8099"
  volumes:
    - ./data/keycloak:/opt/keycloak/data  # volume local sur le systeme
    #- keycloak_data:/opt/keycloak/data   # volume Docker nommé
```

- **port:** permet d'exposer plusieurs ports de conteneurs à une machine parente. Parexemple ports: "3000:8080".
  Elle permet d'exposer le port 8080 au port 3000 de la machine parente.Nous naviguerons ici vers localhost:3000.

- **build :** permet de définir le lien vers le dossier racine contenant le Dockerfile pour réaliser le build

- **image :** permet de définir le nom de l'image à générer

- **depends_on :** permet de définir les autres services dont le service dépend. Nous avons supposé ici que le service candidat dépend du service job. Ceci permet de contrôler l'ordre de démarrage et d'arrêt du service.

Il existe d'autres configurations possibles, telles que volumes, links, command….
[https://github.com/compose-spec/compose-spec/blob/master/spec.md]
[https://docs.docker.com/compose/]

Il faudrait d'autre part changer dans le fichier application.properties du microservice à enregistrer localhost par le nom attribué au serveur de découverte comme suit :

```
eureka.client.server-url.default-zone=http://eureka:8761/eureka/
```

Etape 2: Lancer l'application multi-containers

Pour lancer l'application multi-containers, il suffit d'accéder à l'emplacement du fichier docker-compose.yml et par la suite il faut lancer la commande :

```
docker-compose up
```

NB : Si la commande ci-dessus ne fonctionne pas, vous pouvez utiliser la commande « **docker-compose up -- build**. »,

⇨ C'est la même commande, mais avec le --build on demande d'exécuter les fichiers Dockerfile des micro-services avant d'exécuter le fichier docker_compose.

Il faut avoir le résultat ci-dessous, qui confirme que tous les conteneurs ont été créés.

```
 => [api-gateway internal] load .dockerignore                                                                                    0.0s
 => => transferring context: 2B                                                                                                  0.0s
 => [api-gateway internal] load build context                                                                                    0.0s
 => => transferring context: 83B                                                                                                 0.0s
 => CACHED [api-gateway 2/2] ADD target/Gateway-0.0.1-SNAPSHOT.jar gateway.jar                                                   0.0s
 => [api-gateway] exporting to image                                                                                             0.5s
 => => exporting layers                                                                                                          0.0s
 => => exporting manifest sha256:115cb1ce1fdc5ce862a39ece4916d695761b9a4cf981559bcecb5f6c4290c67c                                0.0s
 => => exporting config sha256:0ae06e24291c722e4e520c9daf02cf6e51fd82a94aed5179a01be035560411c5                                  0.0s
 => => exporting attestation manifest sha256:de1d62d6ec8c8dca9a6793ebe73901928139f4a692bb5a3046decce4178fcaa7                    0.0s
 => => exporting manifest list sha256:7e73559aa3aef7bbc4e9627c492e2e3004ccdad5aa7189d6c08d820ed7597886                           0.0s
 => => naming to docker.io/library/apigatewayms:1.0                                                                              0.0s
 => => unpacking to docker.io/library/apigatewayms:1.0                                                                           0.4s
 => [api-gateway] resolving provenance for metadata file                                                                         0.0s
[+] Running 13/13
 ✓ api-gateway                          Built                                                                                   0.0s
 ✓ candidat                             Built                                                                                   0.0s
 ✓ config-server                        Built                                                                                   0.0s
 ✓ eureka                               Built                                                                                   0.0s
 ✓ job                                  Built                                                                                   0.0s
 ✓ Network microservicealternance_default  Created                                                                              0.3s
 ✓ Container eureka                     Created                                                                                 1.5s
 ✓ Container mysql                      Created                                                                                 1.5s
 ✓ Container keycloak                   Created                                                                                 1.5s
 ✓ Container configserver               Created                                                                                 0.2s
 ✓ Container job                        Created                                                                                 0.2s
 ✓ Container candidat                   Created                                                                                 0.2s
 ✓ Container apigateway                 Created                                                                                 0.2s
```

Vous pouvez aussi vérifier que toutes les images ont été créées sur Docker desktop.

		Name	Tag	Image ID	Created	Size	Actions
☐	●	quay.io/keycloak/keycloak	26.3.1	d9b4b8ff1627	3 months ago	706.13 MB	▷ ⋮
☐	●	mysql	5.6	20575ecebe62	4 years ago	421.86 MB	▷ ⋮
☐	●	eurekams	1.0	937c5d81141f	3 months ago	835.94 MB	▷ ⋮
☐	●	configserverms	1.0	e6634c82c98f	3 months ago	826.42 MB	▷ ⋮
☐	●	candidatsms	1.0	f43f0247ca7b	3 months ago	880.31 MB	▷ ⋮
☐	●	jobms	1.0	1aeb21649a66	4 months ago	879.07 MB	▷ ⋮
☐	●	apigatewayms	1.0	7e73559aa3ae	3 months ago	825.46 MB	▷ ⋮

Showing 7 items

Name	Container ID	Image	Port(s)	Actions
microservicealternance	-	-	-	
mysql	cde562abffb5	mysql:5.6	3306:3306	
keycloak	dc508bd571b3	keycloak/keycloak:26.	8080:8080	
eureka	ff60195b519c	eurekams:1.0	8761:8761	
configserver	db25094df02e	configserverms:1.0	8070:8070	
candidat	0d07b6a92785	candidatsms:1.0	8071:8071	
job	6147aad47aa2	jobms:1.0	8083:8083	
apigateway	301f971dee02	apigatewayms:1.0	9000:9000	

Showing 8 items

Pour appeler les différents services lancés, consulter les liens :

http://localhost:8761/
http://localhost:9000/candidats/
http://localhost:9000/jobs

http://localhost:8080/

Etape 3 : Exportez votre propre relam et l'ajouter dans le keyclock de docker :

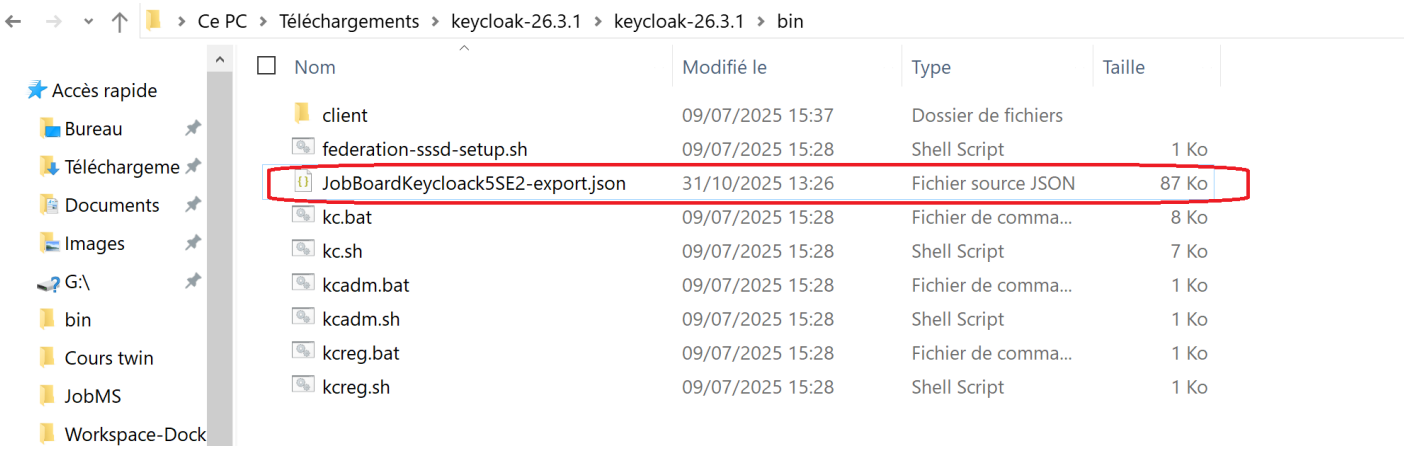
Pour exporter un realm en JSON, **Keycloak doit être arrêté complètement** :

- soit fermer la fenêtre
- soit utiliser : CTRL + C

Lancer l'invite de commande dans le dossier bin de votre répertoire keyclock local et taper cette commande : **kc.bat export –realm nomdevotrerealm --file nomdevotrerealm-export.json**

```
C:\Windows\System32\cmd.exe
Microsoft Windows [version 10.0.19045.6456]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\azer\Downloads\keycloak-26.3.1\keycloak-26.3.1\bin>kc.sh export --realm JobBoardKeycloack5SE2 --file JobBoardKe
ycloack5SE2-export.json

C:\Users\azer\Downloads\keycloak-26.3.1\keycloak-26.3.1\bin>

/usr/bin/bash --login -i C:\Users\azer\Downloads\keycloak-26.3.1\keycloak-2...
Changes detected in configuration. Updating the server image.
Updating the configuration and installing your custom providers, if any. Please
wait.
```

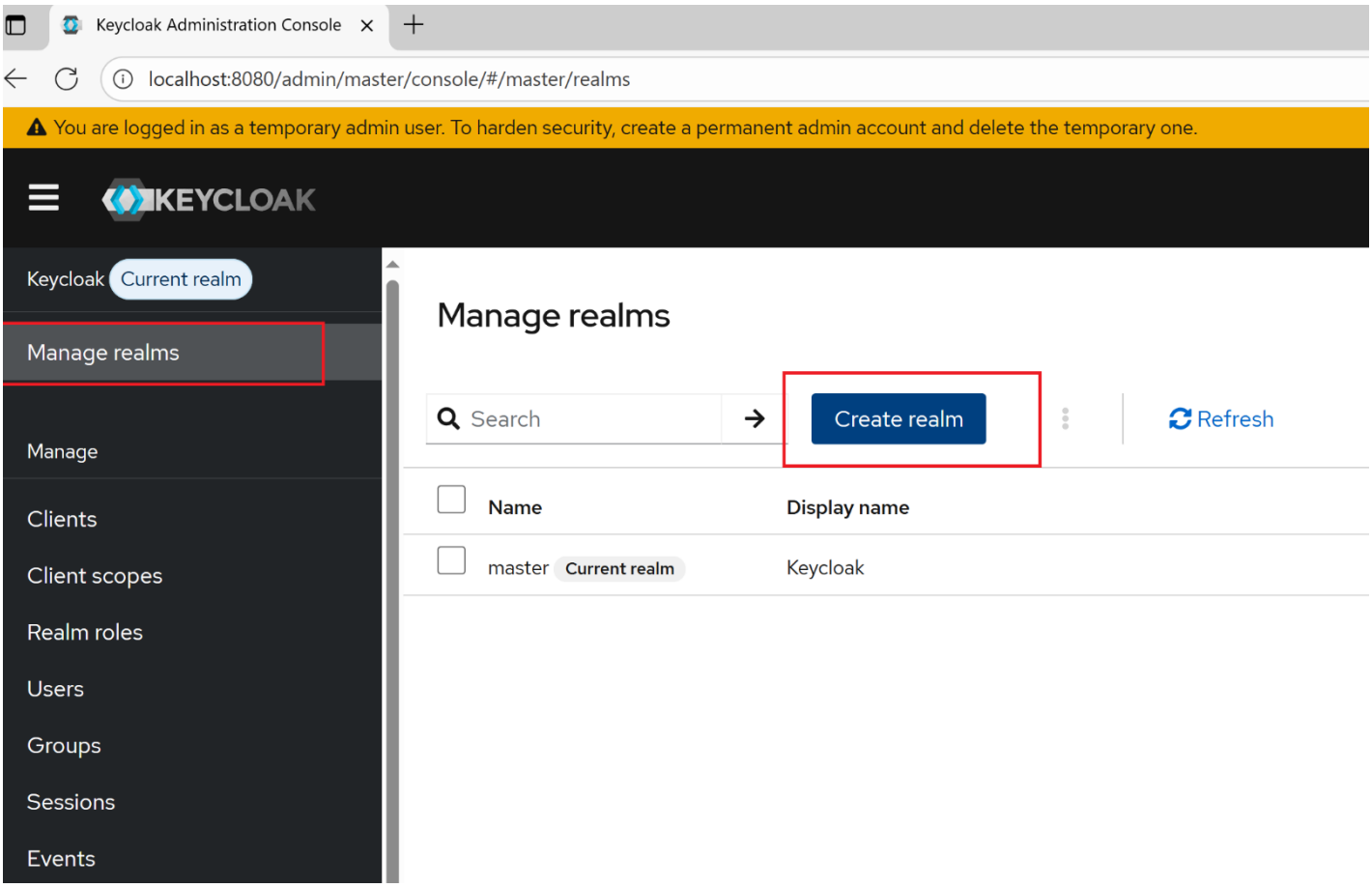
Un fichier json sera créé sous le répertoire bin qui porte nom de votre realm.json :

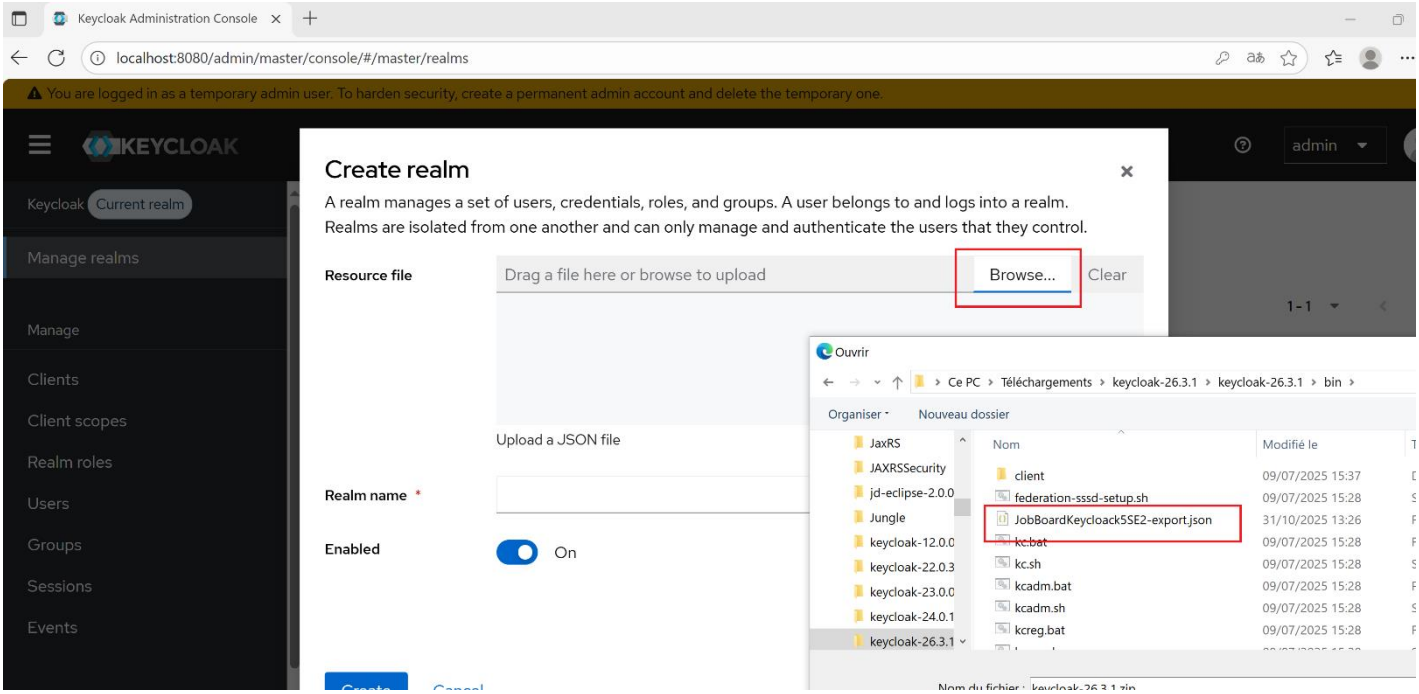


Connectez-vous sur keyclock de docker via http://localhost:8080/ et accéder via le compte :

Username : admin password : admin

Importez votre realm comme indiqué par cette figure create realm->Browse -> garder le même nom pour votre realm:





Etape 4 : Exécuter le fichier docker-compose.yml directement sur docker hub :

On doit lancer les deux commandes

1- **docker compose build** : pour préparer les images.

```
PS C:\Users\Oumeima\Desktop\MicroServiceAlternance> docker compose build
time="2025-10-05T10:30:18+01:00" level=warning msg="C:\\Users\\Oumeima\\Desktop\\MicroServiceAlternance\\docker-compose.yml: the attribute `version` is obsolete, it will be
 ignored, please remove it to avoid potential confusion"
[+] Building 6.1s (33/33) FINISHED                                                                                                                  docker:desktop-linux
 => [eureka internal] load build definition from Dockerfile                                                                                                          0.1s
 => => transferring dockerfile: 155B                                                                                                                                 0.0s
 => [api-gateway internal] load metadata for docker.io/library/openjdk:17                                                                                            2.7s
 => [eureka auth] library/openjdk:pull token for registry-1.docker.io                                                                                                0.0s
 => [eureka internal] load .dockerignore                                                                                                                             0.0s
 => => transferring context: 2B                                                                                                                                      0.0s
 => [eureka internal] load build context                                                                                                                             0.0s
 => => transferring context: 82B                                                                                                                                     0.0s
 => [api-gateway 1/2] FROM docker.io/library/openjdk:17@sha256:528707081fdb9562eb819128a9f85ae7fe000e2fbaeaf9f87662e7b3f38cb7d8                                      0.3s
 => => resolve docker.io/library/openjdk:17@sha256:528707081fdb9562eb819128a9f85ae7fe000e2fbaeaf9f87662e7b3f38cb7d8                                                  0.1s
 => CACHED [eureka 2/2] ADD target/Eureka-0.0.1-SNAPSHOT.jar eureka.jar                                                                                              0.0s
 => [eureka] exporting to image                                                                                                                                      0.3s
 => => exporting layers                                                                                                                                              0.0s
 => => exporting manifest sha256:1b0b40ffb8ce1f96cd90b0d474d5d10f46e2178acf4e251eda76f96dbdc938ec                                                                    0.0s
 => => exporting config sha256:9a0a5f874a00923b86dc1da8268ed0ba25976c422fed0943faeb555a13ea79b6                                                                      0.0s
 => => exporting attestation manifest sha256:412a073216661ad101a9ef7861fec13c484f7f15fc4c7d4e27992573a752906f                                                        0.1s
```

☐	○	oumeimaibnelfekih/eurekams	1.0	5a43a0991ff4	3 months ago	835.94 MB	▷ ⋮
☐	○	oumeimaibnelfekih/configserverms	1.0	dd06b64a83f5	3 months ago	826.42 MB	▷ ⋮
☐	○	oumeimaibnelfekih/candidatsms	1.0	a2b21032cd5c	3 months ago	880.31 MB	▷ ⋮
☐	○	oumeimaibnelfekih/jobms	1.0	67bc8a223a1d	4 months ago	879.07 MB	▷ ⋮
☐	○	oumeimaibnelfekih/apigatewayms	1.0	65bd00fab73a	3 months ago	825.46 MB	▷ ⋮

2- **docker compose push** : pour pousser les images locales vers le Docker Hub

NB : Il est possible que l'exécution de la deuxième commande soit plus longue que d'habitude !

En fin vous pouvez vérifier la création des images sur docker hub.

Exécuter toutes les instances avec KillerCoda :

- ✓ Vérifier que vous êtes bien connecté sur docker Hub
- ✓ Ouvrer la plateforme via ce lien : https://killercoda.com/ puis accéder à **Playgrounds → Ubuntu**
- ✓ On obtient un terminal Linux Docker prêt à l'emploi .
- ✓ Taper la commande : **vi docker-compose.yml** pour créer et ouvrir un fichier docker-compose.yml
- ✓ Copier et coller le contenu de votre fichier docker-compose.yml comme suit :
  - Appuyer sur **la touche *i*** pour passer en mode insertion et écrire dans le fichier ou bien taper la commande : **cat > docker-compose.yml**
  - **Copier** le contenu de votre propre fichier docker-compose.yml **en supprimant toutes les lignes de commandes build :------**

    NB : Dans les plateformes de test cloud , on ne doit pas utiliser la commande build (pas d'images locales à créer). On doit donc modifier le docker-compose.yml pour utiliser **uniquement des images déjà poussées sur Docker Hub**.

  - **Pour coller** le contenu passer par : **ctrl +shift+ V**
  - Pour enregistrer et quitter, appuyer sur **Esc**, puis taper **:wq** et cliquer sur **Entrée** ou bien Terminer avec **Ctrl + D**.
  - Pour vérifier le contenu du fichier dockerj-compose, vous pouvez passer par la commande : **cat docker-compose.yml**

```
KILLRCODA                                   Editor   Tab 1   +

Ubuntu 24.04

This playground is just plain Ubuntu.

START

root@ubuntu:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
root@ubuntu:~$ vi docker-compose.yml
root@ubuntu:~$ cat docker-compose.yml
version: '3.9'

services:

  config-server:
    image: inesmjd/configserver:1.0
    container_name: configServer
    ports:
      - "8888:8888"
    depends_on:
      - eureka-server
    environment:
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka

  job:
    image: inesmjd/job:1.0
    container_name: job
    ports:
      - "8081:8081"
    depends_on:
      - mysqldb
      - eureka-server
      - config-server
    environment:
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
      - SPRING_DATASOURCE_URL=jdbc:mysql://mysqldb:3306/jobTest?createDatabaseIfNo
se&max_allowed_packet=15728640
      - SPRING_DATASOURCE_USERNAME=root
      - SPRING_DATASOURCE_PASSWORD=root

  candidat:
    image: inesmjd/candidat:1.0
```

- Lancer l'exécution via la commande : **docker-compose up -d.** Cette commande téléchargera les images depuis Docker Hub et démarrera les conteneurs.
- Vérifier que les containers ont été bien démarré via la commande **docker ps** :

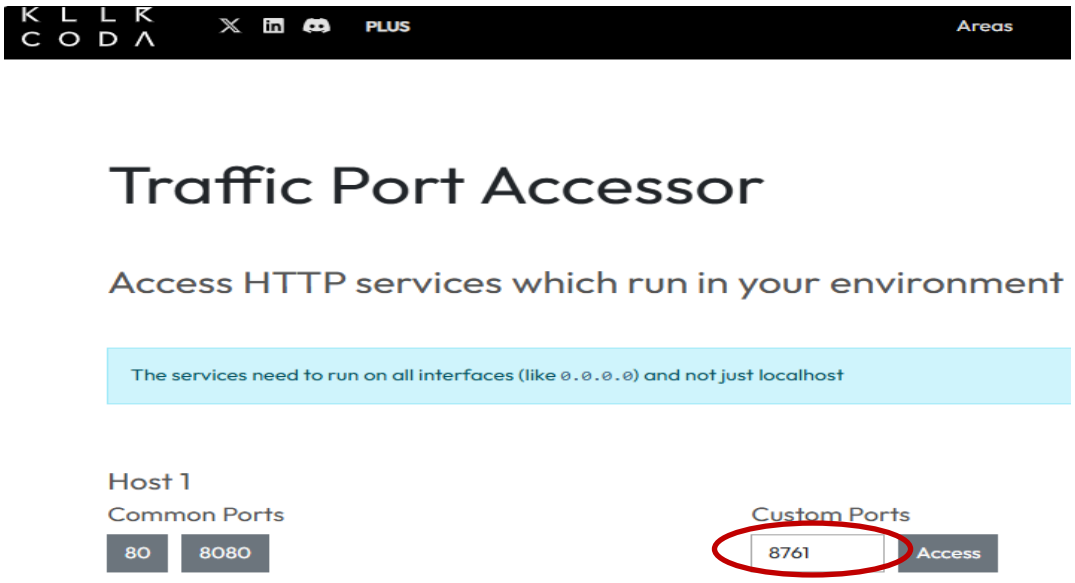
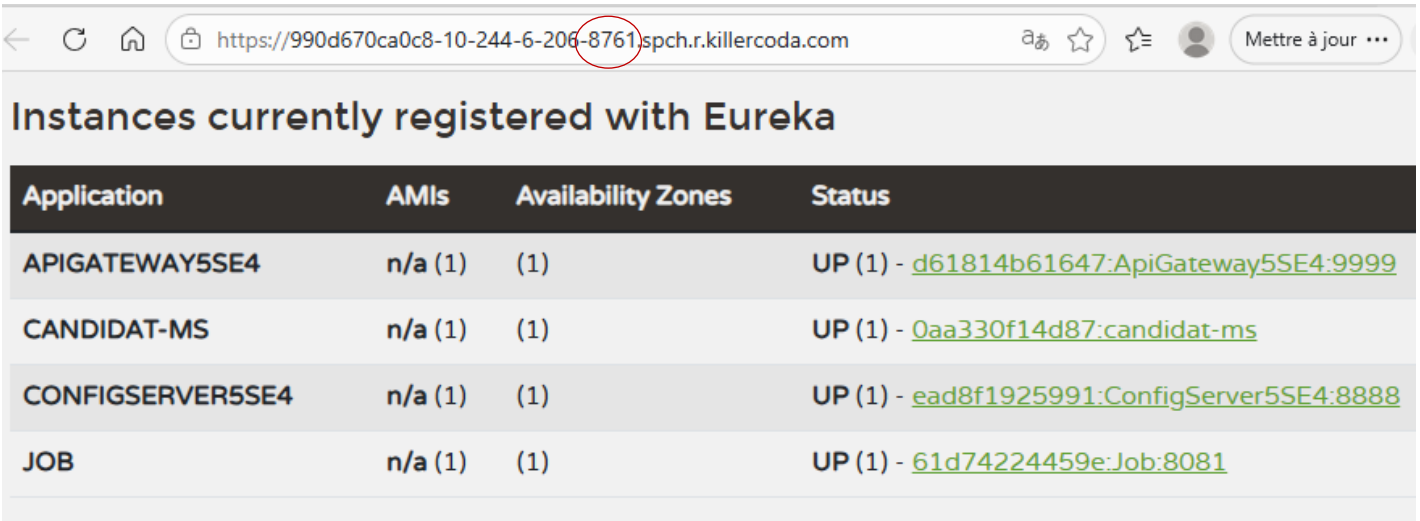
```
root@ubuntu:~$ docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS          PORTS
d61814b61647   inesmjd/apigateway:1.0    "java -jar ApiGatewa…"   8 seconds ago    Up 8 seconds    0.0.0.0:9999->9999/tcp, [::]:9999->9999/tcp
61d74224459e   inesmjd/job:1.0           "java -jar Job.jar"      10 seconds ago   Up 10 seconds   0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
0aa330f14d87   inesmjd/candidat:1.0      "java -jar candidat.…"   10 seconds ago   Up 10 seconds   0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
ead8f1925991   inesmjd/configserver:1.0  "java -jar ConfigSer…"   11 seconds ago   Up 11 seconds   0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
fbde136dc1cf   mysql:5.6                 "docker-entrypoint.s…"   13 seconds ago   Up 12 seconds   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
42bd235f2d4c   inesmjd/eureka:1.0        "java -jar EurekServ…"   13 seconds ago   Up 12 seconds   0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
```

- Tester vos services via KillerCoda en utilisant les ports d'expositions des containers :

  - Cliquer sur le menu (Haut droit), puis choisir **Traffic / Ports** :

```
KILLRCODA   Editor   Tab 1   +                                   32m

Ubuntu 24.04

This playground is just plain Ubuntu.

version: '3.9'

services:

  config-server:
    image: inesmjd/configserver
container_name: configServer2
    ports:
      - "8888:8888"
    depends_on:
      - eureka-server
    environment:
      - EUREKA_CLIENT_SERVICEUR

  job:

New Terminal Window
New Editor Window
Traffic / Ports
Font Size: 1 2 3 4 5 6 7 8
Help / Support
Exit Scenario
```

- Indiquer le port de chaque container à tester via Custom port. Choisir par exemple le port 8761 pour tester l'accès à la page Eureka puis cliquer sur Access



La page suivante s'affichera. Vérifier l'enregistrement des différents services démarrés dans le cloud: